

Component	Test Framework	Total Tests	Status	Line Coverage	Branch Coverage
Backend (Spring Boot)	JUnit 5 / JaCoCo	73	PASSED	76.8%	40.9%
Frontend (React)	Vitest / V8	10	PASSED	79.4%	66.0%
Project Total (Avg)		83	PASSED	78.1%	53.5%

Backend:

Directory Path: `src/test/java/com/cookbook/cookbook/`

- * Total Classes: 27
- * Total Lines: 720 (553 Covered / 167 Missed)
- * Security & Auth: High coverage in SecurityConfig (100%) and AuthController (~90%).
- * Services: RecipeService and UserService have moderate coverage, focusing on logic like forking and rest tokens.

I have successfully executed the tests for both the backend and frontend. The backend tests passed with 76.8% line coverage across 27 classes, while the frontend tests passed with 79.41% line coverage.

CooperCookbook

CooperCookbook

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.cookbook.cookbook.repository		74%		45%	26	67	48	222	16	56	0	6
com.cookbook.cookbook.service		47%		26%	54	70	73	150	34	49	0	4
com.cookbook.cookbook.controller		76%		50%	18	57	40	168	6	41	0	5
com.cookbook.cookbook		81%		16%	4	14	6	32	1	11	0	4
com.cookbook.cookbook.model		100%		n/a	0	104	0	111	0	104	0	6
com.cookbook.cookbook.security		100%		87%	1	12	0	37	0	8	0	2
Total	844 of 3,265	74%	65 of 110	40%	103	324	167	720	57	269	0	27

Test Suite (Class)	Test Cases	Focus Area
AuthControllerTest	8	Login, registration, Step 1 validation
RecipeControllerTest	12	CRUD operations, public/private filtering
RecipeServiceTest	10	Forking logic, ingredient validation
CollectionControllerTest	8	Folder management, adding recipes to collections
UserControllerTest	6	Profile updates, password verification
SavedRecipeControllerTest	5	“Stashing” recipes, removing from stash
JwtFilterTest / JwtUtilTest	8	Token generation, validation, and security
RepositoryTests (Multiple)	15	SQL queries for recipes, ingredients, steps, and users
ModelTest	1	Basic POJO/entity integrity

Frontend:

Directory Path: cookbook-frontend/src/components/___tests___/

- * **Navbar.tsx:** 71.4%
- * **RecipeCard.tsx:** 60.0% (Increased complexity from recent link updates)
- * **TagInput.tsx:** 86.4%
- * **Summary:** The frontend coverage is concentrated on critical UI components and input validation.

All files

80.48% Statements 33/41 66.03% Branches 35/53 64.7% Functions 11/17 79.41% Lines 27/34

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File		Statements	Branches	Functions	Lines	
Navbar.tsx		75%	6/8	50%	4/8	50%
RecipeCard.tsx		60%	3/5	60%	18/30	33.33%
TagInput.tsx		85.71%	24/28	86.66%	13/15	75%

Test File	Test Cases	Focus Area
Navbar.test.tsx	2	Navigation links and active states
RecipeCard.test.tsx	3	Title rendering, author display, and tag truncation
TagInput.test.tsx	2	Adding and removing category tags
TagInputLimits.test.tsx	3	Character limits and duplicate tag prevention

Area	How to View Specific Test Names	Note
Backend	Maven generates a detailed HTML report of every test name and its execution time. Open it with: <code>open target/site/surefire-report.html</code>	If the file does not exist, run: <code>./mvnw surefire-report:report</code>
Frontend	Run UI mode with: <code>cd cookbook-frontend && npx vitest --ui</code>	This opens an interactive browser window where you can click each test file and see the exact <code>it("should...")</code> descriptions.